

Lab 12: Control

Objectives: Control

After completing this lab, you will:

- Learn how to build a simple Control.

Implementing Combinational Control Units

In this section, we show how the ALU control unit and main control unit for the single clock design are mapped down to the gate level. With modern computer aided design (CAD) systems, this process is completely mechanical. The examples illustrate how a CAD system takes advantage of the structure of the control function, including the presence of don't-care terms.

Mapping the ALU Control Function to Gates

Figure 1 shows the truth table for the ALU control function that was developed in the datapath lecture. A logic block that implements this ALU control function will have four distinct outputs (called Operation3, Operation2, Operation1, and Operation0), each corresponding to one of the four bits of the ALU control in the last column of Figure 1. The logic function for each output is constructed by combining all the truth table entries that set that particular output. For example, the low-order bit of the ALU control (Operation0) is set by the last two entries of the truth table in Figure 1. Thus, the truth table for Operation0 will have these two entries.

ALUOp		Funct field						Operation
ALUOp1	ALUOp0	F5	F4	F3	F2	F1	F0	
0	0	X	X	X	X	X	X	0010
X	1	X	X	X	X	X	X	0110
1	X	X	X	0	0	0	0	0010
1	X	X	X	0	0	1	0	0110
1	X	X	X	0	1	0	0	0000
1	X	X	X	0	1	0	1	0001
1	X	X	X	1	0	1	0	0111

Figure 1: The truth table for the four ALU control bits (called Operation) as a function of the ALUOp and function code field

Figure 2 shows the truth tables for each of the four ALU control bits. We have taken advantage of the common structure in each truth table to incorporate additional don't cares. For example, the five lines in the truth table of Figure 1 that set Operation1 are reduced to just two entries in Figure 2. A logic minimization program will use the don't-care terms to reduce the number of gates and the number of inputs to each gate in a logic gate realization of these truth tables.

A confusing aspect of Figure 2 is that there is no logic function for Operation3. That is because this control line is only used for the NOR operation, which is not needed for our RISC-V subset.

ALUOp		Function code fields					
ALUOp1	ALUOp0	F5	F4	F3	F2	F1	F0
0	1	X	X	X	X	X	X
1	X	X	X	X	X	1	X

a. The truth table for Operation2 = 1 (this table corresponds to the second to left bit of the Operation field in Figure C.2.1)

ALUOp		Function code fields					
ALUOp1	ALUOp0	F5	F4	F3	F2	F1	F0
0	X	X	X	X	X	X	X
X	X	X	X	X	0	X	X

b. The truth table for Operation1 = 1

ALUOp		Function code fields					
ALUOp1	ALUOp0	F5	F4	F3	F2	F1	F0
1	X	X	X	X	X	X	1
1	X	X	X	1	X	X	X

c. The truth table for Operation0 = 1

Figure 2: The truth tables for three ALU control lines

Only the entries for which the output is 1 are shown. The bits in each field are numbered from right to left starting with 0; thus F5 is the most significant bit of the function field, and F0 is the least significant bit. Similarly, the names of the signals corresponding to the 4-bit operation code supplied to the ALU are Operation3, Operation2, Operation1, and Operation0 (with the last being the least significant bit). Thus the truth table above shows the input combinations for which the ALU control should be 0010, 0001, 0110, or 0111 (the other combinations are not used). The ALUOp bits are named ALUOp1 and ALUOp0. The three output values depend on the 2-bit ALUOp field and, when that field is equal to 10, the 6-bit function code in the instruction. Accordingly, when the ALUOp field is not equal to 10, we don't care about the function code value (it is represented by an X). There is no truth table for when Operation3=1 because it is always set to 0 in Figure 1.

From the simplified truth table in Figure 2, we can generate the logic shown in Figure 3, which we call the ALU control block. This process is straightforward and can be done with a CAD program. An example of how the logic gates can be derived from the truth tables is given in the legend to Figure 3.

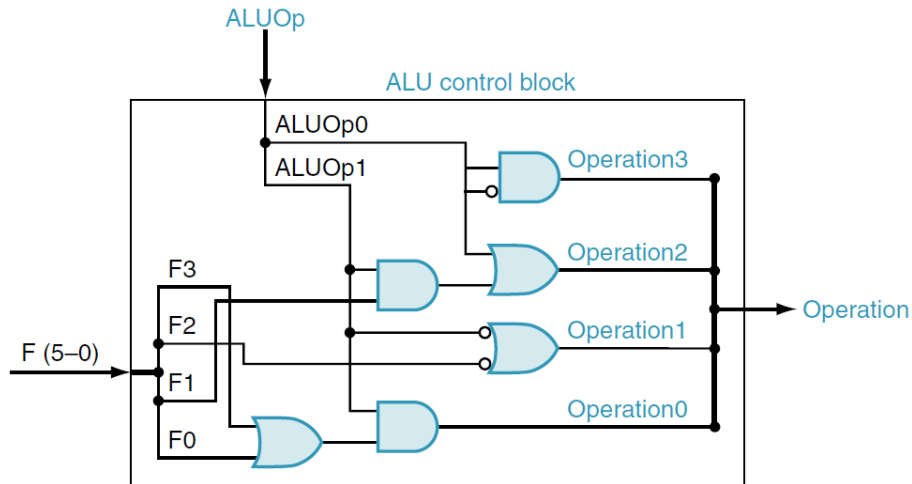


Figure 3: The ALU control block generates the four ALU control bits, based on the function code and ALUOp bits

This logic is generated directly from the truth table in Figure 2. Only 4 of the 6 bits in the function code are actually needed as inputs, since the upper 2 bits are always don't cares. Let's examine how this logic relates to the truth table of Figure 2. Consider the Operation2 output, which is generated by two lines in the truth table for Operation2. The second line is the AND of two terms ($F1 = 1$ and $ALUOp1 = 1$); the top two-input AND gate corresponds to this term. The other term that causes Operation2 to be asserted is simply $ALUOp0$. These two terms are combined with an OR gate whose output is Operation2. The outputs Operation0 and Operation1 are derived in similar fashion from the truth table. Since Operation3 is always 0, we connect a signal and its complement as inputs to an AND gate to generate 0.

This ALU control logic is simple because there are only three outputs, and only a few of the possible input combinations need to be recognized. If a large number of possible ALU function codes had to be transformed into ALU control signals, this simple method would not be efficient. Instead, you could use a decoder, a memory, or a structured array of logic gates.

Mapping the Main Control Function to Gates

Control	Signal name	R-format	lw	sw	beq
Inputs	Op5	0	1	1	0
	Op4	0	0	0	0
	Op3	0	0	1	0
	Op2	0	0	0	1
	Op1	0	1	1	0
	Op0	0	1	1	0
Outputs	RegDst	1	0	X	X
	ALUSrc	0	1	1	0
	MemtoReg	0	1	X	X
	RegWrite	1	1	0	0
	MemRead	0	1	0	0
	MemWrite	0	0	1	0
	Branch	0	0	0	1
	ALUOp1	1	0	0	0
	ALUOp0	0	0	0	1

Figure 4: The control function for the simple one-clock implementation is completely specified by this truth table

Implementing the main control function with an unstructured collection of gates, as we did for the ALU control, is reasonable because the control function is neither complex nor large, as we can see from the truth table shown in Figure 4. However, if most of the 64 possible opcodes were used and there were many more control lines, the number of gates would be much larger and each gate could have many more inputs.

Since any function can be computed in two levels of logic, another way to implement a logic function is with a structured two-level logic array. Figure 5 shows such an implementation. It uses an array of AND gates followed by an array of OR gates. This structure is called a programmable logic array (PLA). A PLA is one of the most common ways to implement a control function.

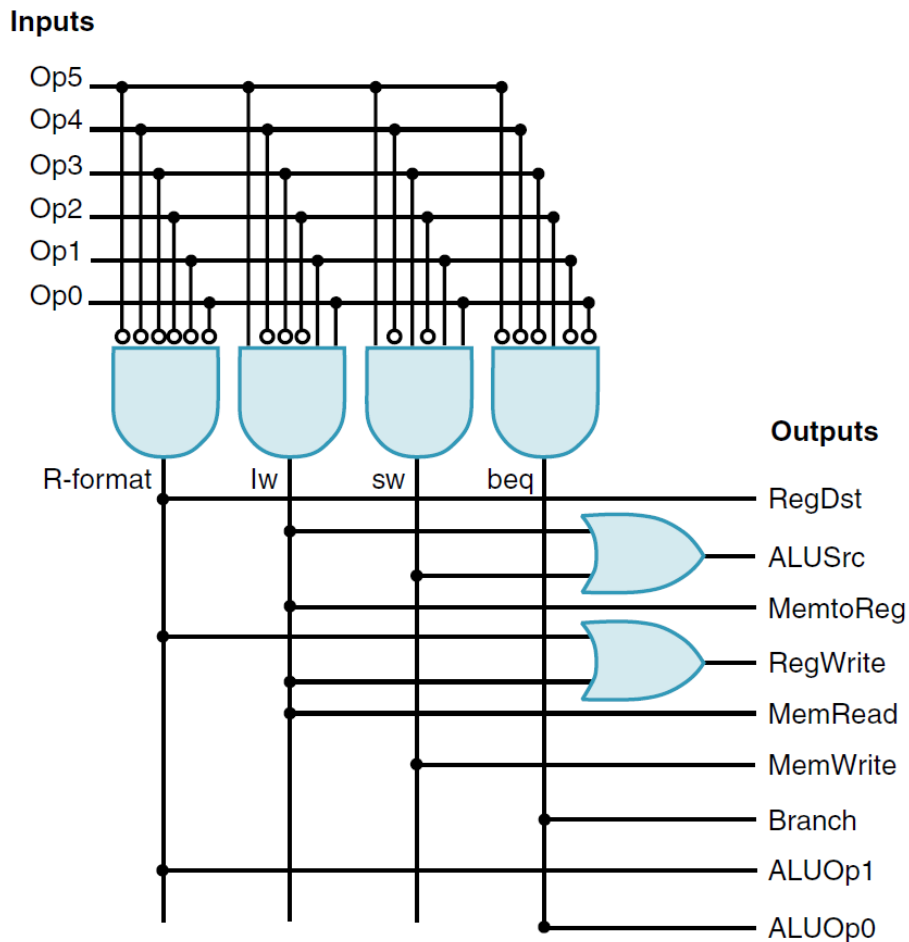


Figure 5: The structured implementation of the control function as described by the truth table in Figure 4

Assignment

1. Build the ALU Control in a sub-circuit called "ALUControl";
2. Build the Main Control in a sub-circuit called "MainControl";
3. **CHALLENGE:** Connect all the sub-circuits you built so far (ALU, Register File, ALU Control, and the Main Control). Then, connect input pins to the Register File read input pins (to provide the numbers of the registers to be read). Similarly, connect a six-bit input pin to the input of the main control. Finally, connect an output pin to the output of the ALU.